

Corporate Policy Assistant

User Chat Query and RAG Response Process

Overview

This process describes what happens after a user enters a Corporate Policy question in the chat interface. The application retrieves relevant policy chunks from Azure AI Search, combines the retrieved information with the user's question and chat instructions, sends the grounded prompt to the Microsoft Foundry LLM, and returns the generated answer to the user.

Important: For vector search, the user's text query must first be converted into a query embedding. Azure AI Search can perform this query-time vectorization through a configured vectorizer that points to the same embedding model used during indexing.

Recommended Query-Time Process

1. User Chat Interface - Submit Policy Question

The user enters a natural-language Corporate Policy question in the React chat interface. Example: "How does the organization measure the severity of an IT incident?"

2. Application / API Orchestration Layer - Receive the Query

The backend application, such as an Azure Function or other API service, receives the user's question. This layer coordinates retrieval, prompt construction, the LLM request, and the final response.

3. Query-Time Vectorization - Convert the Question to an Embedding

If vector or hybrid retrieval is used, the user's text question is sent through the Azure AI Search vectorizer to the Microsoft Foundry embedding deployment, such as text-embedding-3-small. The resulting query vector represents the semantic meaning of the user's question.

4. Azure AI Search - Retrieve Relevant Corporate Policy Chunks

Azure AI Search compares the query against the indexed Corporate Policy content. With vector search, it finds semantically similar policy chunks. With hybrid search, Azure AI Search can combine vector similarity and keyword/full-text search, and semantic ranking can further improve relevance.

5. Azure AI Search Response - Return Grounding Context

Azure AI Search returns the highest-ranked policy chunks and useful metadata to the application. The response can include the policy text, document name, policy identifier, section, page, or other source metadata.

6. Prompt Construction - Combine Chat Prompt + Search Results

The application constructs the grounded LLM request. The prompt typically combines system instructions, the user's policy question, relevant Azure AI Search results, and optionally recent chat history. The retrieved policy content becomes the grounding context for the LLM.

7. Microsoft Foundry LLM - Reason Over the Grounded Context

The application sends the assembled prompt to the deployed LLM. The LLM interprets the user's intent, uses the retrieved Corporate Policy content as evidence, and generates an answer based on that context. The LLM does not search the vector index itself in this classic RAG pattern; retrieval occurs before generation.

8. Application Layer - Process the LLM Response

The backend receives the model response and can apply response formatting, citation/source mapping, logging, safety controls, or other application logic before returning the answer.

9. User Chat Interface - Display the Policy Answer

The React chat interface displays the final grounded answer to the user, optionally including the policy documents or sections used as sources.

Conceptual Architecture Flow

```
Corporate Policy Assistant User
|
v
React Chat Interface
User Policy Question
|
v
Application / API Orchestration Layer
(e.g., Azure Function)
|
v
Query-Time Vectorizer
|
v
Microsoft Foundry Embedding Model
text-embedding-3-small
|
v
Query Vector
|
v
Azure AI Search Service
+-----+
| Corporate Policy Search Index|
| - Text chunks |
| - Vector embeddings |
| - Document metadata |
+-----+
|
v
Relevant Policy Chunks / Search Results
|
v
Application / API Orchestration Layer
Build Grounded Chat Prompt
+ User question
+ Search results
+ System instructions
+ Optional chat history
|
v
Microsoft Foundry LLM
Reason over grounded policy context
|
v
Generated Policy Answer
|
v
Application / API Orchestration Layer
Format / citations / validation
|
v
React Chat Interface
Final Policy Response to User
```

The Key Handoff: Azure AI Search Response + Chat Prompt

A useful way to think about the middle of the RAG process is that **Azure AI Search supplies evidence**, while the **LLM performs language reasoning and answer generation**. The orchestration layer sits between them and combines the two inputs.



Clean Version for Architecture Documentation

Step	Component	Responsibility
1	React Chat UI	Capture and submit the user's Corporate Policy question.
2	Application / API Layer	Coordinate retrieval, prompt construction, LLM invocation, and response handling.
3	Embedding Model / Vectorizer	Convert the user's text query into a query vector for vector or hybrid search.
4	Azure AI Search	Retrieve and rank the most relevant policy chunks and metadata.
5	Prompt Builder	Combine user question, search results, system instructions, and optional chat history.
6	Microsoft Foundry LLM	Reason over the grounded context and generate the Corporate Policy answer.
7	Application / React UI	Format and display the final answer, optionally with policy source citations.

Recommended Simplified Flow

User Policy Question -> Application/API -> Query Embedding -> Azure AI Search -> Relevant Policy Chunks -> Grounded Chat Prompt -> Foundry LLM Reasoning -> LLM Response -> Application/API -> User Chat Response

Microsoft Architecture Notes

Microsoft describes Azure AI Search as supporting vector and hybrid retrieval for RAG workloads. Integrated vectorization can generate query embeddings at query time, and hybrid search can combine vector and full-text queries in the same request. The retrieved content is then used to ground the generative model's response.